

pzip

Komentoja:

```
gcc -o pzip pzip.c -Wall -Werror
```

```
./pzip *tiedosto* + *tiedosto* > *tiedosto*
```

Lyhyt kuvaus

HUOM! Olen käyttänyt tässä aika paljon minun my-zip koodiani.

My-zip ohjelmaa muokattu niin, että työ hajotetaan paloiksi, joita sitten työntekijä-threadit voivat hakea **jonossa**. Työntekijöiden määrä saadaan tehtävänannon vinkin mukaisesti: `get_nprocs()` hyödyntämällä. Haku tapahtuu lukkoa käyttäen, ettei race-condition tapahdu, eli kaksi ota vahingossa samaa työtä. Kun työ on otettu, seuraava saatavilla työ päivitetään ja lukko vapautetaan. Työt käsitellään, tulokset päivitetään structeihin ja main threadissa sitten kootaan näistä yksi kokonaisuus, joka on lopputulos. Eli tässä sovelletaan hajota ja hallitse -menetelmää. Ongelma paloitellaan, sen palat ratkaistaan ja palojen ratkaisusta kootaan itse ongelman ratkaisu.

Toiminnallisuudesta

Ohjelma alkaa taas alustusvaiheella. Parempi ohjelmoija varmaan pärjäisi vähemmälläkin, mutta minä tarvitsin näitä kaikkia. Rivillä 157 tarkistetaan argumenttien määrä, ja jos se on hyväksyttävä, käydään iteroimaan avattavia tiedostoja. Ensimmäinen vaihe on taas niiden avaaminen lukemista varten (read only), ja jos se ei onnistu, siitä ilmoitetaan ja poistutaan `exit(1)`-hyödyntäen.

Jos avaaminen onnistuu, edetään tiedoston koon määrittelyyn ja tarkistukseen, ettei koko ole 0, eli tyhjä. Sitten määritellään työn koko jakamalla tiedoston koko työntekijöiden määrällä. Tämän jälkeen hyödynnetään ohjeissa ehdotettua `mmap()`-funktia `fread:in()` sijaan. `Mmap()` kartoittaa tiedoston muistiosoitteisiin, joten se ei toimi täysin samalla tavalla kuin `fread()`.

`Mmap()` jälkeen, määritellään työmäärä, ja tässä on tärkeä havaita se, että ei tehdä työmäärää tasan jaolliseksi työntekijöiden määrän perusteella, koska silloin töitä olisi jokaiselle työntekijälle yksi kappale, jolloin jono olisi turha ja ei olisi mitään hyötyä nopeammista työntekijöistä. Tämmöisessä tilanteessa ”heikoin lenkki” eli hitain prosessori määräisi täysin työtahdin, kun `pthread_join()` odottaa sen työn valmistumista. Sen sijaan annoin töille kertoimen, jolloin töitä on työntekijöiden määrä * kerroin. Käytin kerrointa 5. Tämä mahdollistaa sen, että nopeat prosessorit voivat käydä hakemassa enemmän kuin yhden työn ja tehdä ne pois alta, vaikka hitaampi prosessori työstää yhtä työtä koko työn ajan. Eli tehokas työnteke palkitaan lisätyöllä, niin kuin yrityksissä.

Seuraavaksi rivillä 204 alustetaan lukko, jota hyödynnetään myöhemmin työpalasten haussa. Myös muistia varataan tarvittava määrä ja merkkilaskuri alustetaan (rivi 225). Riveillä 229-232 alustetaan tiedot tietueeseen, jonka jälkeen määritellään työ, työjärjestys ja aloitetaan määrittelemään työntekijöitä ja ne alkavat työstämään.

Siirrytään tässä vaiheessa `tWorker()` funktioon ja vasta myöhemmin palataan tähän 240 riville. `tWorker()` määrittelee mitä työntekijä tekee. Eli tänne tulee iso osa siitä, mitä my-zipissä tehtiin mainissa yhdessä Main Threadissa. Aluksi alustetaan muuttujat, sen jälkeen aloitetaan silmukka, joka tarkistaa boolean (ei oikea tyyppi, mutta ideana 1 ja 0), että työtä on tarjolla.

Jos töitä on tarjolla, otetaan lukko, otetaan työpalanen työstettäväksi, ilmoitetaan/päivitetään seuraavan työn olevan nykyinen työ + 1 paikassa, ja avataan lukko. Rivillä 87 tarkistetaan, ettei ole viimeinen työ tai mennyt viimeisen yli, jos ehto täyttyy, niin päivitetään, ettei työtä ole tarjolla. Muuten otetaan työpalanen structista, määritellään sen alku ja loppupiste, merkin tiedot ja alustetaan muuttujat (uudestaan, koska silmukan sisällä). Rivillä 99 tehdään toistoja alkupisteestä loppupisteeseen asti. Otetaan nykyinen merkki ja tehdään aivan sama käsittely, kuin my-zipissä: Jos on alkuvaihe, edellinen merkki on nykyinen merkki (tämä rakenne oli opetettu ohjelmoinnin perusteet -kurssilla). Jos merkki on sama, kasvatetaan sen laskuria yhdellä. Jos merkit ovat eri, tallennetaan tietueeseen/structiin (en pidä sanasta "tietue", joten käytän tästä edespäin vain struct). Päivitetään edellinen kirjain nykyisellä ja asetetaan merkkimääräksi 1, koska se ei mene enää iCharacterCount == 0, joka on vain ensimmäistä kierrosta varten. Jos näistä ehdoista mikään ei tapahdu, se menee else haaraan ja ilmoittaa huolimattomuusvirheestä. Viimeinen merkki käsitellään silmukan ulkopuolella.

Nyt palataan riville 243, jossa pthread_join() odottaa silmukan sisällä. Kyseinen funktio odottaa, että kaikki työntekijät ovat valmiina. Kun kaikki ovat valmiina, Main Thread alkaa kokoamaan niiden työtuloksista lopputulosta. Ensiksi otetaan nykyisen merkin tiedot muuttujaan, jonka kautta voidaan sitten iteroida nykyisten merkkien verran my-zip tyylisesti. Otetaan nykyinen merkki ja sen määrä, jos merkkimäärä on 0, edellinen merkki on kyseinen nykyinen merkki ja edellisen merkin määrä on nykyisen merkin määrä. Jos merkit ovat samat, eli käsitellään vielä samaa merkkiä, edellisen merkin määrään lisätään yksittäisen merkin määrä. Jos merkit poikkeavat toisistaan, kirjoitetaan määrä ja merkki ulostuloon stdout. Lopuksi edellinen merkki ja sen määrä päivitetään. Else-haara on taas vain paljastamaan ajatusvirheitä / huolimattomuusvirheitä. Tästä muodostuu lopullinen tulostiedosto.

Lopuksi vapautetaan muisti, suljetaan avatut tiedostot ja kirjoitetaan vielä viimeinen merkki silmukan tiedot silmukan ulkopuolella. Rivillä 294 on vielä else ehto virheellisten syötemäärien tulostukseen. Lopuksi vielä ohjelmasta poistuminen return(0) kautta.

Testaus ja toiminnallisuus

Automaattitestit (pitää uudelleen nimetä ja muuttaa tulosteet pzip:stä wzipiksi):

```
root@LEO-PC:~/Käyt Sys/ostep-projects/initial-utilities/wzip# ./test-wzip.sh
test 1: passed
test 2: passed
test 3: passed
test 4: passed
test 5: passed
test 6: passed
```

Muistitesti useammalla tiedostolla:

```
root@LEO-PC:~/Käyt Sys/Projekti5# valgrind --leak-check=full --show-leak-kinds=all ./pzip testi1.txt testi2.txt > testi.z
==94796== Memcheck, a memory error detector
==94796== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==94796== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==94796== Command: ./pzip testi1.txt testi2.txt
==94796==
==94796==
==94796== HEAP SUMMARY:
==94796==    in use at exit: 0 bytes in 0 blocks
==94796==   total heap usage: 200 allocs, 200 frees, 24,384 bytes allocated
==94796==
==94796== All heap blocks were freed -- no leaks are possible
==94796==
==94796== For lists of detected and suppressed errors, rerun with: -s
==94796== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
root@LEO-PC:~/Käyt Sys/Projekti5# ./pzip
pzip: file1 [file2 ...]
```

```
root@LEO-PC:~/Käyt Sys/Projekti5# ./pzip virheellinentiedosto.txt
pzip: cannot open file
```

[illegible]

punzip

Komentoja:

```
gcc -o punzip punzip.c -Wall -Werror
```

```
./punzip *tiedosto* + *tiedosto* > *tiedosto*
```

Lyhyt kuvaus

HUOM! Olen käyttänyt tässä aika paljon minun my-unzip ja pzip koodiani, koska ohjelmoiija on aina laiska - sääntö ja ei ole järkevää tehdä samoja asioita eri tavoilla vain, koska ohjelma on eri. Melkein kaikki Main Threadin toiminnallisuus on sama kuin pzipissä, muutamaa muuttujaa ja structia lukuun ottamatta. Structien poikkeus aiheuttaa myös muutoksia muistin varauksessa, mutta nämä ovat loppujen lopuksi hyvin samanlaisia ohjelmia. Yksi ohjelma tekee yhdellä tavalla, ja toinen käänteisellä. Työt kuitenkin jaetaan palasiksi ja niitä haetaan jonossa lukon avustuksella. Käyn kuitenkin toiminnallisuudesta-osioissa kaiken uudestaan läpi, ettei jää mitään epäselvyyksiä tai huomiotta.

Toiminnallisuudesta

Ohjelma alkaa riviltä 104, jonka perässä on heti muuttujien alustus. Sen jälkeen tarkistetaan argumenttien määrä, ja jos niitä on tarpeeksi, mennään for silmukkaan, luettavien tiedostojen määrän verran. Avataan tiedostoja open()-funktiolla (ei fopen()), ja jos se epäonnistuu, ilmoitetaan tulosteella ja poistutaan exit(1) käyttäen.

Jos avaus onnistuu, määritellään tiedoston koko, ja jos se ei ole tyhjä/0, määritellään kartoitettu tiedosto hyödyntäen mmap()-funktiota, kuten tehtävänannossa kehoitettiin. Seuraavaksi lasketaan 4 tavuisen numeron ja 1 tavuisen merkin koko, jota hyödynnetään seuraavaksi parimäärän laskemisessa, jakamalla tiedoston koko parikoolla. Työmäärä saadaan taas työntekijöiden määrällä * työpalasten määrä kertoimella, mahdollistaen jonorakenteen. Tarkistetaan, että onko työmäärä suurempi kuin työpalasten määrä, ja jos on, työmäärä on palasten määrä.

Seuraavaksi lasketaan työn koko, ideana poistaa jakojäännöksen ylijäämä ja sen tuloksen avulla päivitetään vielä työmäärä. Rivillä 158 aletaan määrittämään työpalasten alku ja loppupisteitä. HUOM! pzipissä olevassa linkkilähteessä mainitaan, että jos tämän tekee järkevästi, niin ei tarvitse lukkoa lainkaan. Halusin kuitenkin käyttää lukkoa, koska sitä tässä opetettiin. Viimeinen työpala on järkevä vain asettaa viimeiseen pisteeseen, koska silloin se ei mene vahingossakaan yli (rivit 162-163). Sitten laitetaan structiin alku ja loppupisteet, jotta niitä voi myöhemmin hyödyntää. Rivillä 171 alustetaan lukko, kuten pzipissä.

Rivillä 174 aloitetaan muistin varaaminen, en tiedä mitä siinä erikoisesti kertoisi. Tarvitaan muistia, joten sitä varataan. Hyödynnetään työn alku ja loppupisteitä. Kaikki niin kuin C-kielen kurssilla opetettiin. Rivillä 201 asetetaan/alustetaan structin arvot, jonka jälkeen rivillä 207 silmukassa määritellään työ, työjärjestys ja luodaan/määritellään työntekijät ja pistetään ne töihin. Tässä nyt siirrytään samalla tavalla kuin pzipin selosteessa tWorker()-funktioon, jossa käsitellään mitä työntekijät tekevät.

tWorker() alku on muuttujien alustusta, kuten koulun ohjelmointityyli opetti. Niiden jälkeen alkaa while-silmukka, joka toimii niin kauan kuin töitä on tarjolla. Lukitaan saatavilla olevat työt, otetaan sieltä työpalanen työntekijälle itselleen, päivitetään/ilmoitetaan seuraavan työn olevan otetusta työstä seuraava ja avataan lukko. Tarkistetaan, tarvitseeko päivittää työsaatavuus ja jos ei, jatketaan rivin 76 else-haaraan.

Haetaan structista työpalanen, sen alku ja loppupiste ja kun useampi työntekijä voi käsitellä samaa merkkiä otetaan myös heidän tuloksensa. Rivillä 84 aletaan työstämään työpalasta alusta loppuun. Aluksi

määritellään tavupalanen askelten määrän ja koon perusteella, jonka jälkeen alustetaan silmukassa vielä muuttujat. Kopioidaan tarvittava muisti, ja otetaan käsiteltävä merkki. Tämän jälkeen ihan niin kuin my-unzipissä, käsitellään yksittäinen merkki ja sen määrä.

Nyt palataan riveille 215-217, jossa Main Thread odottaa työntekijä-threadien työn valmistumista. Tämän jälkeen työn tulokset kootaan ja kirjoitetaan juuri niin kuin my-zipissä. Tämän jälkeen vapautetaan käytetty muisti, pointterit asetetaan NULLiksi (vaikka ei tarvitsisi, näin meille opetettiin C-kielen kurssilla). Lopuksi vielä tiedostot suljetaan. Munmap() voisi kyllä tehdä aiemmin, mutta haluan tehdä samat toiminnot aina samassa paikassa. Rivillä 237 on else-haara syötteiden puutteelle ja lopussa on vielä ohjelman loppu, return(0).

Testaus ja toiminnallisuus

Automaattitestit (pitää uudelleen nimetä ja muuttaa tulosteet pzip:stä wzipiksi):

```
root@LEO-PC:~/Käyt Sys/ostep-projects/initial-utilities/wunzip# ./test-wunzip.sh
test 1: passed
test 2: passed
test 3: passed
test 4: passed
test 5: passed
test 6: passed
```

Muistitesti useammalla tiedostolla:

```
root@LEO-PC:~/Käyt Sys/Projekti5# valgrind --leak-check=full --show-leak-kinds=all ./punzip testi.z testi1.z > testi000.txt
==28682== Memcheck, a memory error detector
==28682== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==28682== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==28682== Command: ./punzip testi.z testi1.z
==28682==
==28682==
==28682== HEAP SUMMARY:
==28682==    in use at exit: 0 bytes in 0 blocks
==28682==   total heap usage: 243 allocs, 243 frees, 23,600 bytes allocated
==28682==
==28682== All heap blocks were freed -- no leaks are possible
==28682==
==28682== For lists of detected and suppressed errors, rerun with: -s
==28682== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Liian vähän argumentteja:

```
root@LEO-PC:~/Käyt Sys/Projekti5# ./punzip
punzip: file1 [file2 ...]
```

Virheellinen syöte:

```
root@LEO-PC:~/Käyt Sys/Projekti5# ./punzip virheellinensyöte.txt
punzip: cannot open file
```

Testitulos tiedosto:

```
testi0000.txt
1  This is a test
2  is something else
3  a banana
4  test file
5  file test
6  and something
7  these are
8  are these
9  its a question
10 contents leaked
11 Theeere aaaare aaalso
12 Looooonger wooordddssThere are
13 different
14 Test files
15 With different
16 Words
17 There are
18 different
19 Test files
20 With different
21 Words
22 Maybe too
23 many test
24 files hmm?
```